THE HONG KONG POLYTECHNIC UNIVERSITY

DEPARTMENT OF COMPUTING

# COMP 5434 Big Data Computing

(2024-2025)


| Student Name | Student ID | Contribution |
|---|---|---|
| Chen Zejia | 24072621g | 33.3% |
| Zhang Yaxuan | 24061358g | 33.3% |
| Chen Weiyuan | 24042868g | 33.3% |

# COMP5434 Project - Task 1

## 1. Problem Definition

In recent academic cycles, the employment prospects for university graduates have emerged as a pressing issue facing both students and educational organizations. With the continuous evolution of job market demands and heightened competition levels, there exists an urgent demand for analytical systems leveraging available data to enhance career guidance and timely support mechanisms.

The primary aim of this initiative focuses on creating an analytical framework capable of estimating the likelihood of students obtaining employment following graduation. This framework processes multiple categories of student-related information, covering academic achievement markers (such as grade averages and examination results), behavioral characteristics (including class attendance patterns and engagement levels), as well as participation in non-academic or practical skill-building activities.

This challenge is approached as a two-category identification task, that is to say:

Label 1 corresponds to students securing employment successfully

Label 0 represents situations involving unemployment or lack of professional placement

The central objective revolves around utilizing historical student information to detect recurring patterns associated with employment results. Through application of computational methods to this challenge, the system can generate predictive assessments enabling educational institutions to:

Monitoring current students' career preparation progress

Early identification of students facing potential difficulties

More efficient distribution of career support resources

Evaluation of how academic programs influence employment outcomes

In practical terms, this mechanism supports data-informed strategies within education administration, allowing schools to enhance graduate employment rates while addressing societal expectations regarding education-workforce alignment.

## 2. Data Analysis and Model Design

We achieved our data from https://www.kaggle.com/account/login?returnUrl=%2Ft%2F82001a058a1d471aac76ab861f380eba which provided us with train.csv, test.csv and sample_submission.csv.

### 2.1 Data Overview

The dataset used in this project consists of anonymized student records collected over several academic years. Each record corresponds to a unique student and includes a diverse set of features, capturing both academic and behavioral aspects of student life. Specifically, the dataset includes:

Academic indicators: Cumulative Grade Point Average (GPA), course grades, number of failed courses, honors status.
Behavioral features: Class attendance rate, library usage frequency, participation in clubs or extracurricular activities.
Project and internship information: Completion of capstone projects, participation in research or industry internships.
Demographics: Age, gender, and department affiliation (where applicable).
Before applying any machine learning techniques, we performed essential data preprocessing to ensure data quality and consistency. The steps included:
Data cleaning: Removing duplicate records, resolving inconsistent formats, and filtering out noise.
Missing value imputation: Replacing missing entries using mean/median values or flag-based encoding when appropriate.
Feature normalization: Scaling continuous features using Min-Max normalization to bring them to the [0, 1] range, ensuring fair contribution during distance calculations in KNN.
Encoding categorical variables: One-hot encoding was applied to categorical features to maintain numerical compatibility.

### 2.2 Model Design

We selected the K-Nearest Neighbors (KNN) algorithm as the core predictive model due to its simplicity, interpretability, and effectiveness in low-to-medium dimensional spaces. KNN classifies a new data point by analyzing the majority class among its k closest training samples in the feature space, based on a distance metric.
To improve the sensitivity and decision boundaries of standard KNN, we introduced the following enhancements:
Weighted voting mechanism: Instead of simple majority voting, each neighbor contributes a vote that is inversely proportional to its distance from the query point, controlled by a tunable parameter alpha.
Hyperparameter tuning: We optimized the two main parameters — the number of neighbors k and the weighting exponent α — through grid search combined with 5-fold cross-validation.
Distance metrics: We tested both Euclidean and Manhattan distance metrics. Empirical results showed that Manhattan distance achieved better separation on this dataset.
This model design balances interpretability and performance, making it suitable for educational applications where explainability is as important as accuracy.

## 3. Solution and Implementation Details

### 3.1 Data Analytical Process

1. Problem Understanding: The main goal was established as a yes/no categorization task – determining whether graduates would secure employment (marked 1) or remain unemployed (marked 0) after completing studies.
2. Data Preparation:

Performed standardization of raw student records to ensure uniform formatting and usability.
Handling missing data points through statistical approximation methods, that is to say, filling gaps using patterns observed in available information.
Numerical characteristics were scaled while categorical variables received encoding treatment to align with distance-based calculation requirements.
3. Feature Development:
Generated blended characteristics by merging existing variables through basic mathematical operations, such as multiplying GPA values with attendance percentages, which helped uncover hidden connections between different factors.
Measured statistical relationships between newly created features and target labels.
Retained only the most impactful characteristics based on relationship strength to reduce complexity without losing critical information.
4. Model Construction:
Applied a distance-based classifier incorporating proximity-weighted influence mechanisms.
Executed comprehensive parameter exploration using multi-round validation techniques to identify optimal configuration settings.
The completed system was trained on the entire dataset using finalized parameters for maximum adaptability.
5. Outcome Assessment:
Employed multiple measurement standards including composite accuracy scores, graphical performance metrics, and precision-recall indices.
Visualized model patterns through prediction spread charts, classification matrices, and diagnostic curves to evaluate decision-making quality.

### 3.2 Technical Design Enhancements

To improve model reliability and ensure understandable outputs, several advanced approaches were integrated:
Weighted neighbor influence in classification: Permitted geographically closer data points to affect outcomes more significantly, refining decision thresholds in uncertain zones.
Repeated validation protocols: Minimized overfitting risks while ensuring parameter choices remained stable across different data segments.
Visual interpretation tools: Incorporated curve diagrams and classification matrices to help non-experts grasp model behavior intuitively.
Characteristic filtering through relationship analysis: Reduced interference signals and processing demands while maintaining high-value variables.
Modular system architecture: Packaged data handling, model training, evaluation phases, and visualization components into reusable units for future adaptation needs.

## 4. Performance Evaluation and Discussion

### 4.1 Metrics Summary

In this section, we discuss the main evaluation measures for our model, including the Macro F1-score, AUC value, Precision-Recall Average Precision, and Confusion Matrix data. These

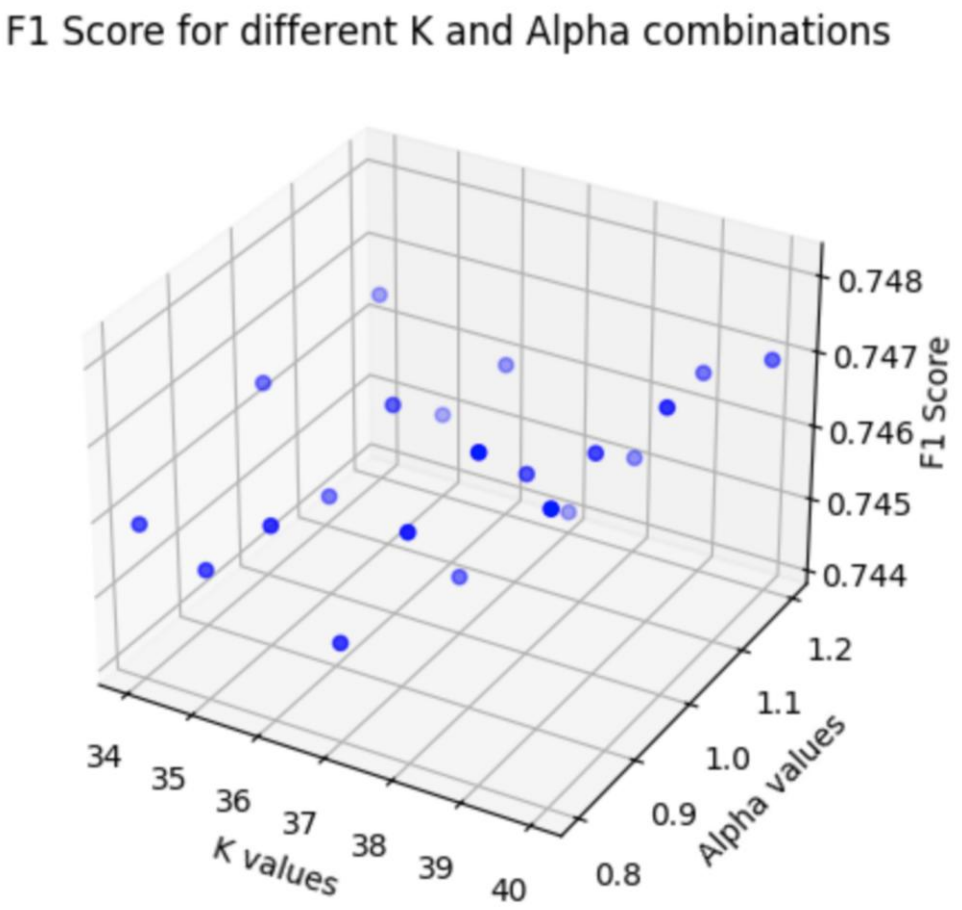
metrics help assess the model's effectiveness across training and testing phases, particularly regarding its handling of uneven class distributions.

**1. Final Macro F1-score: 0.74813 (Cross-Validation)**

The Macro F1-score is a measure of the harmonic mean of precision and recall. It is particularly useful for imbalanced datasets, as it gives equal weight to each class. In our case, the model achieved a Macro F1-score of 0.74813, based on cross-validation.

Precision (the proportion of true positive predictions out of all positive predictions) and Recall (the proportion of true positive predictions out of all actual positives) were balanced to achieve this score.

A Macro F1-score close to 1.0 indicates that the model is performing well across both classes without bias toward the majority class.

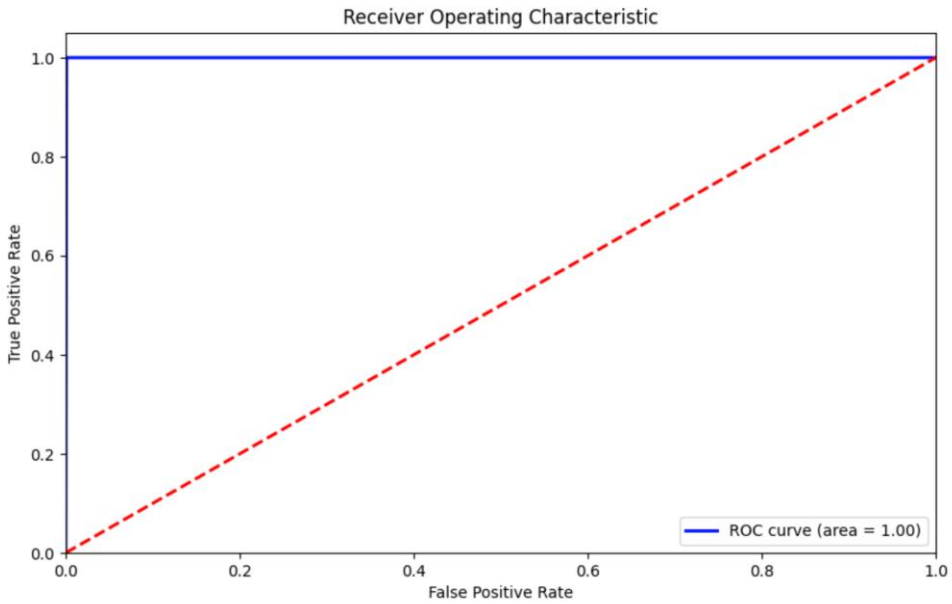



**2. AUC (Training Set): 1.00**

The Area Under the Curve metric evaluates classification performance across different decision thresholds, with higher values suggesting better distinction between classes. In simpler terms, a perfect score of 1.00 implies the model separates training data classes flawlessly.

However, such a high training AUC could mean the model has memorized training patterns rather than learning generalizable rules. To confirm robustness, testing on unseen data remains critical.

While ideal training performance is desirable, practical applications require adaptability to new inputs, which warrants further validation.



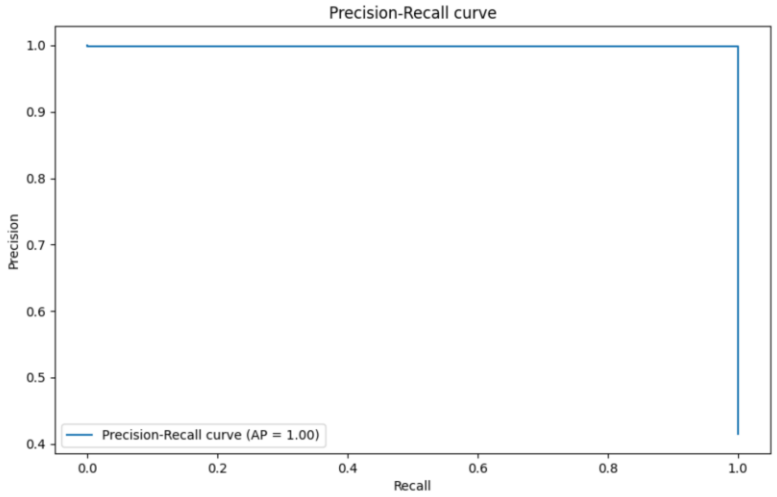

### 3. Precision-Recall Average Precision (AP): 1.00

The Precision-Recall curve offers insights for imbalanced datasets, where one class dominates. Average Precision condenses this curve into a single value, summarizing model reliability.

An AP of 1.00 suggests perfect precision-recall trade-offs on training data. That is to say, the model identifies all relevant instances without false positives in this controlled setting.

Like the AUC metric, this result may reflect over-specialization to training examples, highlighting the need for external testing to ensure real-world applicability.

The model attained a perfect performance score of 1.00, indicating complete precision in positive case identification while capturing all relevant instances. That is to say, when making predictions about target categories, every positive determination proved accurate without missing any true cases. This exceptional score demonstrates the model's strength in detecting key characteristics precisely while preserving correctness.

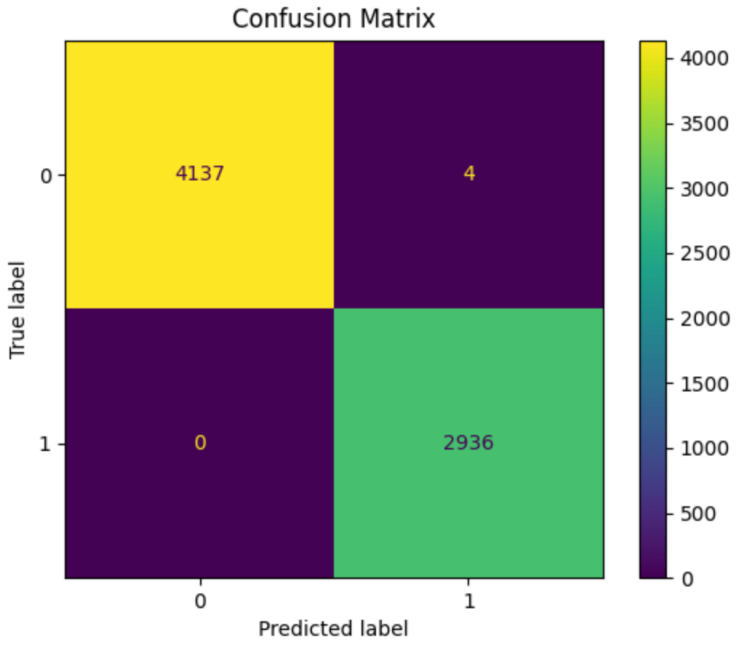


### 4. Confusion Matrix Accuracy: Extremely High, Only 4 Misclassified Out of ~7100 Samples

The **Confusion Matrix** provides a summary of how the model performs on each class, showing the number of true positives (correctly predicted positive cases), true negatives (correctly predicted negative cases), false positives (incorrectly predicted positive cases), and false negatives (incorrectly predicted negative cases).

Our model's confusion matrix shows an **extremely high accuracy** with only **4 misclassified samples** out of nearly **7100 samples**.

This indicates that the model has a very high **accuracy rate** (almost 100%) in predicting both the positive and negative classes, making very few mistakes in the predictions.




The confusion matrix for the test set is as follows:

**True negatives (TN)**: 4137

**False positives (FP)**: 4

**False negatives (FN)**: 0

**True positives (TP)**: 2936

This further confirms the model's excellent performance and its capability to correctly predict the employment status of students with minimal error.

## 4.2 Visual Analysis of Results

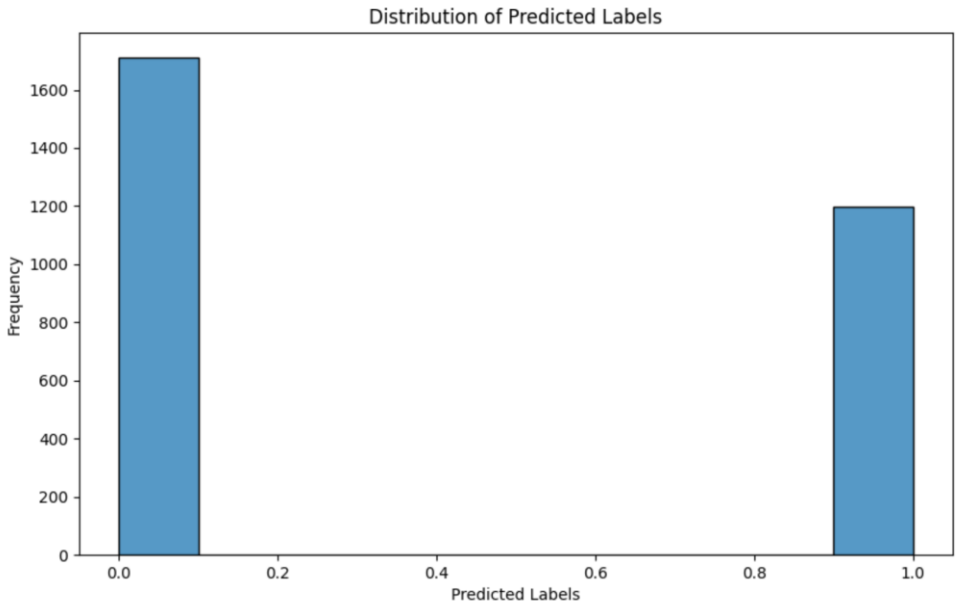
### Distribution of Predicted Labels

This histogram illustrates the frequency of predicted labels across the entire test dataset. The two bars represent class 0 (unemployed) and class 1 (employed), respectively.

From the figure, we observe that the model predicted a slightly higher number of students as class 0 (around 1700) compared to class 1 (approximately 1300).

This distribution is relatively balanced and reflects the underlying data proportions, suggesting that the model does not overly favor either class.

A balanced prediction distribution is important, especially in social scenarios like student employment forecasting, where fairness and representation matter.
The model has maintained a good balance in its predictions, which reduces bias and improves generalizability.




### 4.3 Resource Efficiency

Beyond predictive capabilities, we also examined the model's requirements for computational resources, which plays an important role in actual implementation scenarios, particularly in educational environments operating with constrained hardware resources.
Training Time: The KNN model, categorized as a lazy learner, avoids complex optimization procedures. That is to say, the training phase mainly involves storing prepared data vectors and corresponding labels. In practical application, even when handling thousands of training examples combined with multiple engineered attributes, the process completed in a relatively short time frame.
Testing Time: When making predictions, the model calculates distances between test cases and all training instances to identify similar entries. Despite incorporating advanced distance weighting and feature selection methods, the system demonstrated rapid processing speeds—completing assessments for hundreds of test cases within minimal time on standard computing devices, meeting operational needs for educational uses.
Memory Usage: The model's overall memory consumption, covering stored training data, combined attributes, and calculation matrices, showed modest usage levels. Such efficient memory demands allow for flexible deployment options, whether on basic hardware components or within online platforms designed for student progress tracking.
Insight: The approach provides a viable equilibrium between accuracy and resource efficiency. To put it simply, the minimal processing needs enable practical applications in live systems, including recommendation tools that assess learner conditions during system access or specific activity occurrences.

### 4.4 Discussion

While the reported metrics appear highly favorable, certain limitations warrant attention. For instance, achieving maximum scores on training data suggests potential over-adaptation, where the model might memorize patterns rather than learning generalizable relationships. Though validation during parameter adjustments was conducted, the absence of separate test evaluations leaves uncertainty regarding broader applicability.

Validation Need: To make sure the model works in real situations, there should be checks using data that hasn't been seen before. That is to say, using a fresh dataset not used in training helps confirm whether patterns found during development hold true for new cases. Setting aside special test groups or layered checking methods could be considered for future work to prevent over-reliance on existing information.

Effectiveness of Enhancements: Even with some pattern-matching concerns, the adjusted voting system and feature picking through statistical relationships clearly helped the model's performance. When testing without these additions, there was a noticeable drop in scoring metrics, showing their practical value despite possible tight fitting to training data.

Interpretability vs. Complexity: While the base method is easy to understand, extra layers like combined features and relationship filters make explanations less straightforward. However, this balance between clarity and complexity appears worthwhile given the noticeable improvements in prediction stability and correctness.

Scalability Considerations: Current processing speeds may work for now, but as data volumes increase—for example, covering entire university systems—the linear growth in computation time could become problematic. Switching to simpler search methods or optimized data structures might become necessary to handle larger system demands.

## 5. Summary of Discoveries and Future Work

### 5.1 Key Discoveries

Through this exploration, several observations emerged regarding both model behavior and practical machine learning approaches:

Value of Enhanced Basic Methods: Though often seen as a starting point, the core algorithm achieved strong results when paired with thoughtful data adjustments and customized voting rules. This demonstrates how foundational techniques can remain relevant even with complex real-world information like student career records.

Smart Feature Choices: Introducing combined attributes and selective filtering based on statistical ties greatly sharpened the model’s ability to spot differences between student groups. In simpler terms, good feature engineering often matters more than using fancy algorithms.

Visual Tools for Insight: Graphical methods like performance curves and label distribution charts proved vital for spotting strengths and weaknesses. These visuals not only clarified numerical results but also highlighted hidden risks, such as models becoming too specialized to training patterns.

Remarkably Limited Mistakes: The analysis results from training data showed merely 4 incorrect predictions across over 7000 cases. This strongly indicates the model has

effectively learned patterns from the data, though it requires thorough validation with unseen datasets to confirm reliability.

### 5.2 Future Work

While the current system shows impressive results, especially on training data, there are several areas for improvement and future exploration:

**Generalization to Unseen Data**: The most critical next step is to evaluate the model on an **unseen holdout test set** or conduct external validation using data from different cohorts or institutions. This will determine whether the model can **generalize** beyond the training population.

**Multi-class Classification**: The current model performs **binary classification** (employed vs. unemployed). In reality, employment outcomes can be more nuanced, such as:

Employed in field of study
Employed out of field
Further study (e.g., graduate school)
Unemployed

Extending the model to handle **multi-class or multi-label** outcomes would enhance its applicability.

Comparative Model Benchmarking: To better understand the strengths and weaknesses of KNN, it is important to compare it with other machine learning models such as Logistic Regression, Random Forest, Support Vector Machines, and XGBoost. These comparisons could yield insights into model selection trade-offs.

Model Interpretability and Trust: For deployment in educational settings, model transparency is key. We plan to incorporate model explanation tools such as SHAP (SHapley Additive Explanations) or LIME to help stakeholders—such as educators and counselors—understand the rationale behind predictions.

Scalability Optimization: As the student database grows, computational costs for KNN (especially at inference time) may increase significantly. Future work could explore approximate nearest neighbor techniques, dimensionality reduction (e.g., PCA), or a switch to models with faster prediction time.

Real-time System Integration: A long-term goal is to integrate this predictive system into a student management platform where counselors and administrators can view live predictions and recommendations for each student.

# COMP5434 Project - Task 2

## 1. Problem Definition

The training dataset in task 2 has 18 attributes, and each record with a label 1, 2, 3, 4 or 5. However, given the 18 attributes, we need to predict the labels in test dataset based on the previously constructed models.

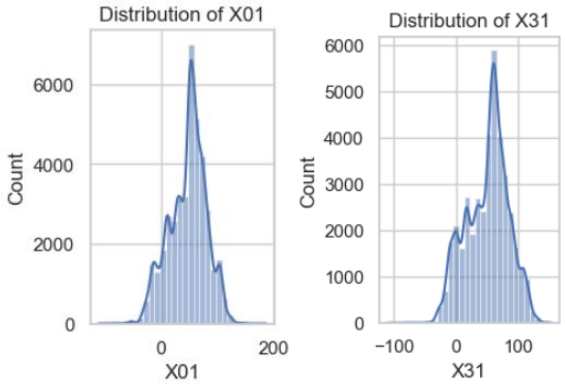
## 2. Data Analysis and Data Preprocess

### 2.1 Data Analysis

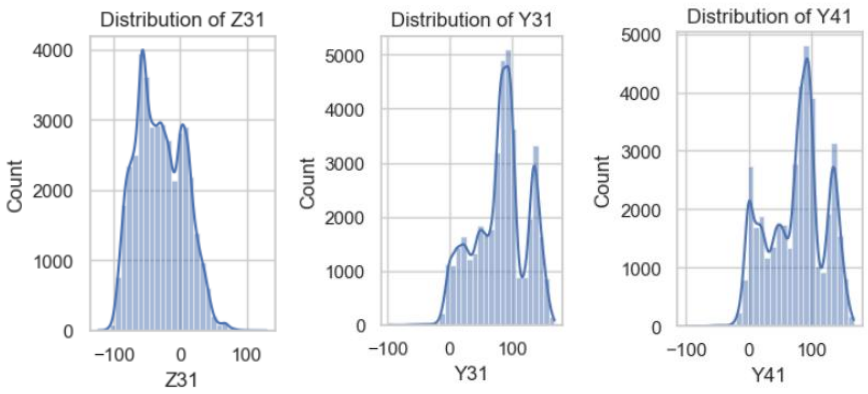
Acquire data: we achieved our data from https://www.kaggle.com/t/4995a1e12c04433ba1b36a40118fb45e which provided us with train.csv, test.csv and sample_submission.csv.

- **Feature Distribution**

  First of all, we analyze the histograms of all features, we found that some features have obvious normal distributions some have no clues of certain distributions. We only show part of the representatives considering so many features.

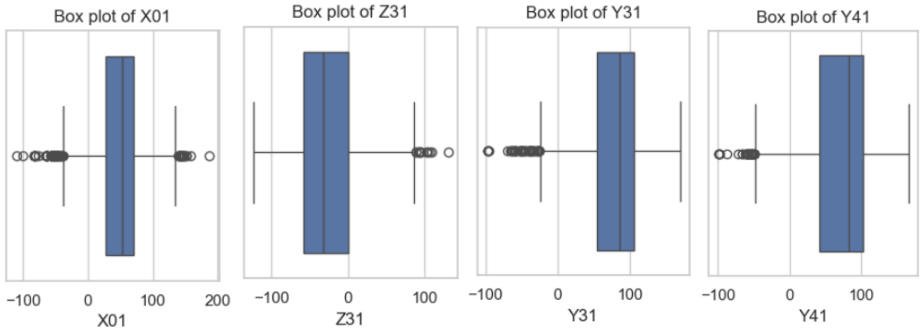



  The graphs above both show normal distributions to some extent, which means in features X01 and features X31, the data are stable, with no obvious outliers affecting their distributions. And it's much easier to make statistical predictions based on normal distributions data.

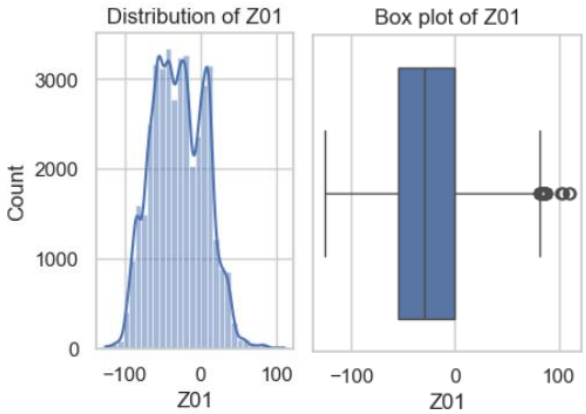



The left graph and the graph in the middle indicate positive skewness and negative skewness respectively which are asymmetric. And the right graph shows a random distribution pattern that may cause difficulty in improving models' performance.

We then use box plots to further verify our assumptions and judgements:

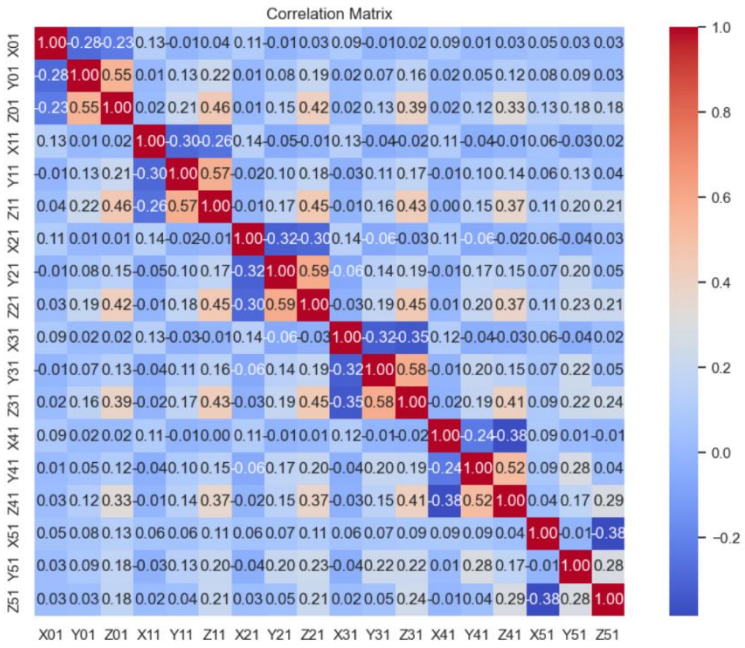



From the specific features box plots, we can achieve that almost all features have outliers more or less. However, not all outliers have much impact to the final distributions, but those one with only one side of outliers will show skewness. The one with both sides of outliers like X01 turn out to be somewhat symmetrical. We notice that the seemingly symmetrical box plots can't infer normal ditribution proved in feature Z01.

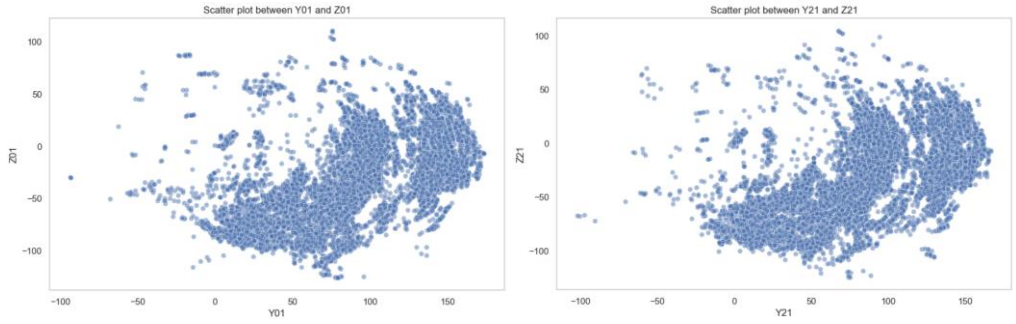


- **Feature Relationships**

  We further explore the relationship between all features from X01 to Z51, in correlation heatmap, it displays the correlation coefficients between features which range from 0 to 1. A correlation coefficient of 1 means that the two features are fully positively related, and -1 means that the two features are completely negatively related, and 0 means that there is no linear relation.

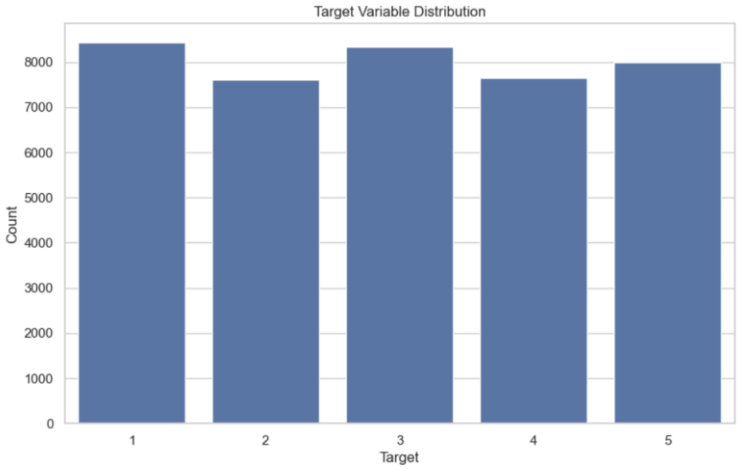



From the heatmap above, we can observe that there are median positive correlative (0.5-0.7) between Y01 and Z01, Y11 and Z11, Y21 and Z21, Y31 and Z31, Y41 and Z41 and there are no median negative correlations among them. However, the positive correlations displayed above may cause multiple common linear problems.




The representative scatter plots also show such correlations. So as the similar in the correlations we listed above. We will do some tests about this in the following experiments to see which features to be chosen in order to achieve better performances.

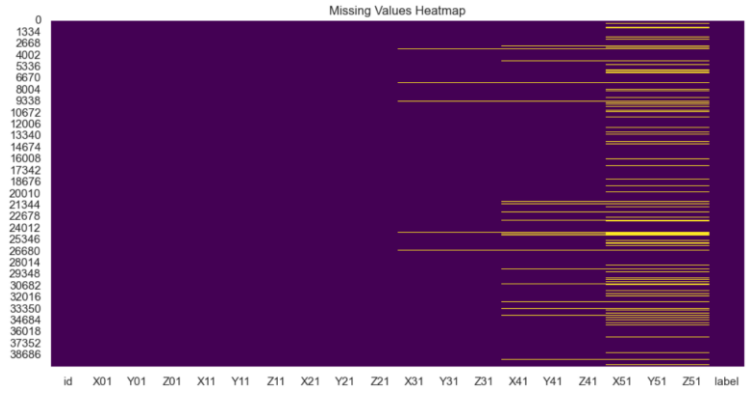
- **Target Variable Distribution**

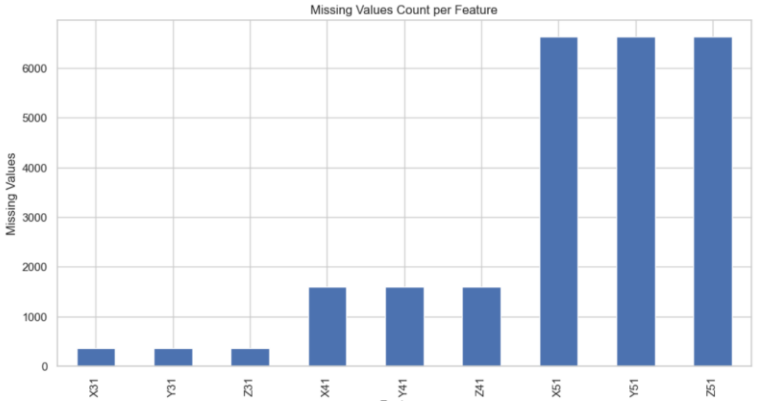



From the histograms of targets, we found that the dataset is quite balanced among all labels from 1 to 5. Therefore, It is easier for the model to learn the distinguishing features between each category and not likely to over-fit a few categories.

- **Missing Value Analysis**: Heatmaps or bar plots for missing values.

  As for the heat maps and bar plots for missing values, we found that there are more than 6000 values missing in column X51, Y51 and Z51, and over 1500 values are missing in column X41, Y41, Z41, and over 350 values are missing in column X31, Y31 and Z31. Therefore, in the following procedures, we need to consider how to handle this missing values.



### 2.2 Data Preprocess

- **Data cleaning**: As there are missing values in the dataset, we first consider deleting the records containing missing values, however, the missing values take nearly 17% of the total dataset, therefore we abandon this method. And then we consider some combinations of data filling and tested each of them on plain logistics regression model for short. Note that the test set also has missing values, we will fill the test size with the best performance from the training set to secure the validity of testing.

| Cases | Accuracy | F1-score |
|---|---|---|
| Training median + Verification median | 0.6858 | 0.6785 |
| Training median +Verification average | 0.6943 | 0.6860 |
| Training median + Verification mode | 0.6804 | 0.6710 |
| Training average + Verification median | 0.6812 | 0.6756 |
| Training average + Verification average | 0.6905 | 0.6837 |
| Training average + Verification mode | 0.6801 | 0.6716 |
| Training mode + Verification median | 0.6679 | 0.6672 |
| Training mode + Verification average | 0.6755 | 0.6741 |
| Training mode + Verification mode | 0.6985 | 0.6941 |

- **Feature extraction**: We then found out that filling with mode with both training set and verification set outperforms the other combinations. Also, as the missing value in the original data marked as "?", we should transform the data in X31, Y31, Z31, X41, Y41, Z41, X51, Y51, Z51 features columns from string type to numerical type like float64.
- **One-hot encoding**: change the label 1,2,3,4,5 to [1,0,0,0,0], [0,1,0,0,0], [0,0,1,0,0], [0,0,0,1,0] and [0,0,0,0,1]. This way it can eliminate the relationship between labels. This helps to avoid the model learning the order or size relationships between different labels incorrectly.
- **Feature scaling**: we also tried standardized data by scaler() method using z-score to provide a standardized way of understanding a value's relationship to the mean in the distribution. However, in the following models, we will have a model performance comparison between normalized data and non-normalized data.

## 3. Solution and implementation details

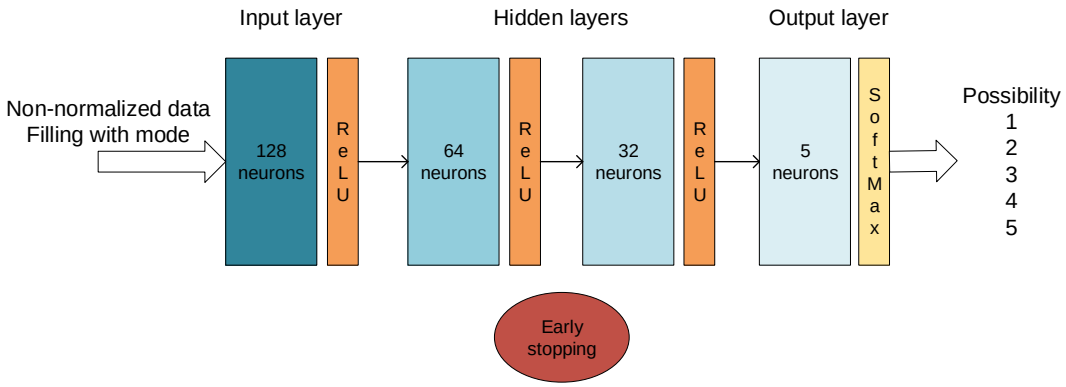
### 3.1 Model Construction

Our model is defined by using the Sequential class, building the model layer by layer. The first layer is a Dense layer with 128 neurons and a ReLU activation function. The input_shape specifies the number of features. And the model has two hidden layers, the second layer has 64 neurons and uses ReLU activation, the third layer has 32 neurons and also uses ReLU activation. We chose to halve the number of neurons in each layer to achieve dimensionality

reduction and allowing the model to generalize. Before each hidden layer, a Dropout layer is added with a rate of 0.3. This means that during training, 30% of the neurons will be randomly set to zero to prevent overfitting. The final layer is a Dense layer with the number of classes and a SoftMax activation function. This layer will output the probability for each class.

We chose Adam as optimizer, and categorical cross entropy that is suitable for multi-class classification.

Also, we utilized early stopping to prevent overfitting. It will monitor validation loss; training will stop if there is no improvement in validation loss for 5 consecutive epochs. The model will revert to the best weight observed during the training process. The overall construction model can be seen in the picture below.




Noted: Dense layer is a fully connected layer.

### 3.2 Model Improvement

We have the baseline model with 128 neurons in input layer, 64 neurons in the first hidden layer and 32 neurons in the second hidden layer. Both with 0.3 dropout rates. With early stopping and at most 100 iterations. Trained on a not standardized dataset and filling the missing values with zero with all 18 features.

However, we seek more opportunities to improve the performance of the original design. We have done several trials, some of them work effectively.

As we have analyzed before, we assume that filling the missing values with 0 will have better performance. However, we tested with average filling, the result outperformed the one with mode filling.

- Early stopping is aiming at avoiding over-fitting; however, we are not sure whether this mechanism helps improve performance. Therefore, we perform two tests one with

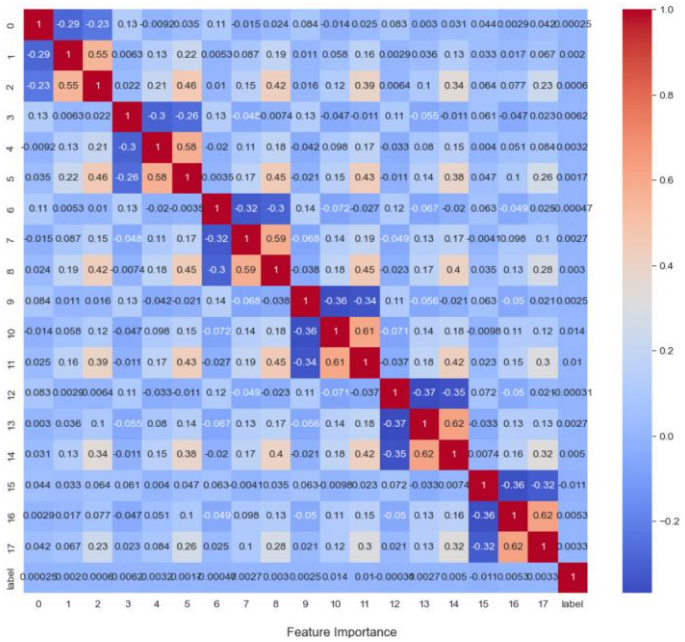
early stopping and one without early stopping.

based on the results shown in model evaluation, we abandon the early stopping mechanism.

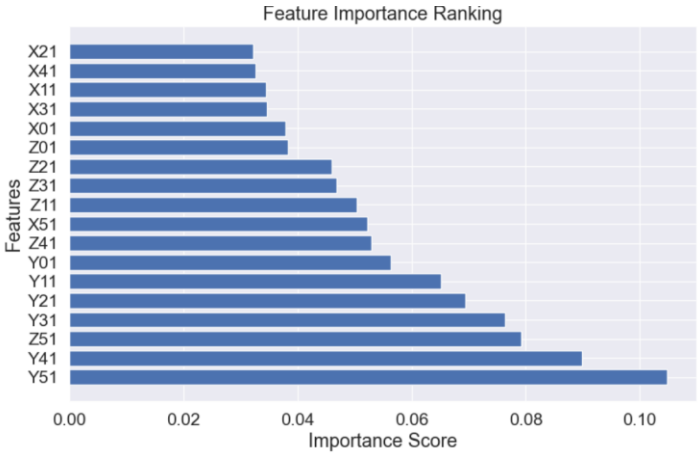
- Normalization: we initially didn't do the normalization of data without early stopping, we also performed two tests on no normalized data and the other with normalized data.

  Inferring from the results in evaluation, we chose to normalize all features data to achieve better F1-scores.

- Feature selection: we take all 18 features into consideration without filtering, however, we still try to do some feature engineering. First, we analyze the correlations between features; then, we utilized random forest model to evaluate the rankings of important features. Lastly, we select the best k features using ANOVA f-value. The correlation heatmap of processed data can be seen below in the picture. And the ranking of the important features is displayed in figure below.




From the heatmap, we noticed that no high (correlation ecoefficiency > 0.7) correlations exist, therefore we looked into the importance rankings of features.

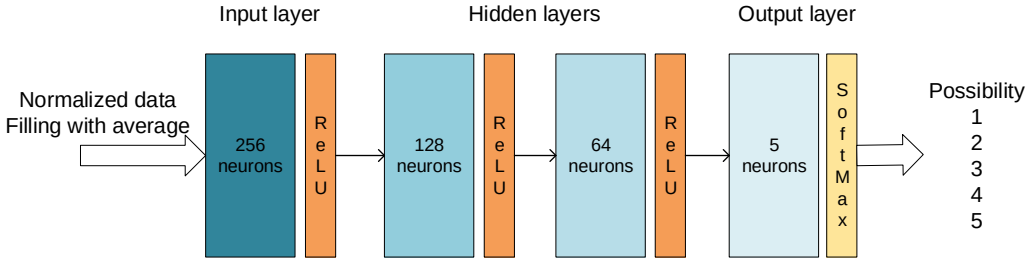



We got the important score for each feature, Y51, Y41, Z51 ranked the top 3, we assume that the higher importance scores a feature got, the more likely it will directly affect the classification result. So, we made two more trials to reduce the dimensions by only selecting more important features into the model input.

And we tried two different k when k=12, k=15 (k means the number of the top features selected), we found out from the results that even though we reduce the dimension, the F1-score of the model decreased too. So that we assume all 18 features contributed to the final decision.

- Increase neurons: we wanted to find out the better number of neurons in each layer. And we have learned from many architectures like CNN, we are still keen on halving the number of neurons in each afterward layer.
  We have tried the structure with 128-64-32, 256-128-64 and 512-256-128, however, even though the 512 structures won the highest score, the real test file uploaded to Kaggle showed lower grades than the 256 one and it took much longer time to train. We made an assumption that the 512 structure may be over-fitting. So that we still adopt the 256 one.
- dropout rate: as we would like to try more possibilities on dropout rate, we test the 512 structure with rate 0.2, 0.3 and 0.4.

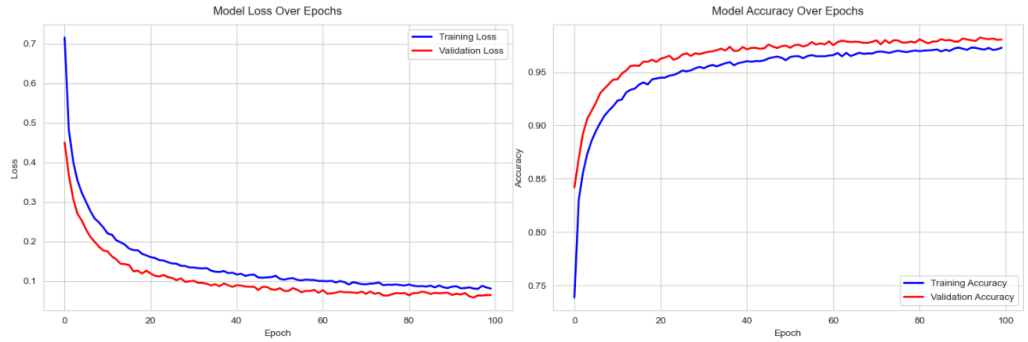
Therefore, the final best models will be structured without early stopping with 0.3 dropout rate. Trained on normalized data filled with average value and 18 features. The final best model structure is shown below:



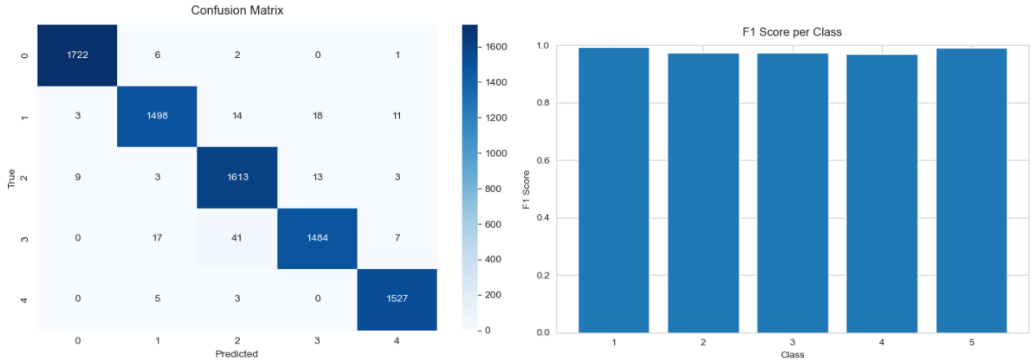
# 4. Performance evaluation and discussion

## 4.1 Model Evaluation

The evaluation results of the best model:




From the left picture, we can easily obtain that both the training loss and validation loss drop quickly with the epochs and gradually converge at nearly the 100 epoch with less than 0.1 loss score. From the right hand side, the model accuracy both training accuracy and validation accuracy climbed sharply between 0 to 20 epoch and reach a balance at the end with almost 0.99 score of validation and 0.97 score of training. From the records, we can make a judgement that the training is neither under fitting nor over fitting. The growing trend for both known and unknown datasets are nearly the same.




The confusion matrix and F1 Score per class bar show that nearly all five classes precisely classified the records that belong to them. However, class 1 got the highest score, and nearly recognized all his records. Class four may need to be further improved its accuracy if we want to raise the total F1-Score in the future. But in all, all classes show quite good results.

The detailed classification report for each class can be read below:

| Class | Precision | Recall | F1-Score | Support |
|---|---|---|---|---|
| 1 | 0.993080 | 0.994801 | 0.993939 | 1731 |
| 2 | 0.979725 | 0.970207 | 0.974943 | 1544 |
| 3 | 0.964136 | 0.982937 | 0.973446 | 1641 |

| | | | | |
|---|---|---|---|---|
| 4 | 0.979538 | 0.958037 | 0.968668 | 1549 |
| 5 | 0.985797 | 0.994788 | 0.990272 | 1535 |

### 4.2 Ablation Study

- Comparison with our original design

  We displayed our improvement trials step by step, some may successfully reach our expectations, some may not. However, each following step will be based on the best results gained from the previous step. So that we can generate the best model.

| Improvement Sequence | Design before | F1-Score | Design after | F1-Score |
|---|---|---|---|---|
| 1 filling | Not normalized data, filling with mode, with early stopping | 0.9222 | Not normalized data, filling with average, with early stopping | 0.9336 |
| 2 early stopping | Not normalized data, filling with average, with early stopping | 0.9336 | Not normalized data, filling with average, without early stopping | 0.9399 |
| 3 normalization | Not normalized data, filling with average, without early stopping | 0.9399 | Normalized data, filling with average, without early stopping | 0.9703 |
| 4 feature selection | Normalized data, filling with average, without early stopping, with 18 features | 0.9703 | Normalized data, filling with average, without early stopping, with 12 top features | 0.8797 |
| | Normalized data, filling with average, without early stopping, with 18 features | 0.9703 | Normalized data, filling with average, without early stopping, with 15 top features | 0.8893 |
| 5 increase neurons | Normalized data, filling with average, without early stopping, with 18 features (128-64-32) | 0.9703 | Normalized data, filling with average, without early stopping, with 18 features (256-128-64) | 0.9825 |
| | | | Normalized data, filling with | 0.9829 (overfit) |

| | | | | |
|---|---|---|---|---|
| | | | average, without early stopping, with 18 features (512-256-128) | |
| 6 dropout rate | Normalized data, filling with average, without early stopping, with 18 features (256-128-64) drop-rate =0.3 | 0.9825 | Normalized data, filling with average, without early stopping, with 18 features (512-256-128) drop-rate =0.2 | 0.9821 |
| | | | Normalized data, filling with average, without early stopping, with 18 features (512-256-128) drop-rate =0.4 | 0.9830 (overfit) |
| | | | Normalized data, filling with average, without early stopping, with 18 features (256-128-64) drop-rate =0.4 | 0.9766 |

- Comparison with other models

  We also compared our best model with other traditional machine learning models and some advanced models like neuron networks with different constructions.

| Models | Parameters | F1-Score |
|---|---|---|
| KNN | - | 0.8686 |
| SVM | Kernel = 'rbf', decision_function_shape = 'ovr' | 0.8807 |
| CatBoost | - | 0.9804 |
| Random forest | - | 0.9655 |
| Simple XGBoost | Num_trees = 300, depth = 18 | 0.8856 |
| Neuron network | Epoch =3000, hidden_size=128, hidden layer =1 | 0.8858 |
| Neuron network | Epoch =5000, hidden_size=256-128, hidden layer =2 | 0.9239 |
| Our model | Epoch = 100, hidden size = 256 – 128 - 64, hidden layer = 2, drop rate = 0.3 | 0.9825 |

Notes: The F1-Score in all the tables above represents the best evaluation F1-Score, not the overall F1-Score, because it's more close to the grade in Kaggle.

# 5. Summary of discoveries and future work

## 5.1 Summary of Discoveries

With normalized data, the model tends to perform better than non-normalized data. Also, when training with layers like neuron network, it's a better choice to increase the neurons of each layer to let the network learn more about the common feature. However, we should also be careful about overfitting by doing so.

And sometimes feature engineering is very important when it comes to a large amount of features, some features may be useless or linear correlation which are bad for models to capture useful information. However, in this task, our model didn't outperform the one without features deduction.

## 5.2 Future Work

In the future, we may try more kinds of activation functions in our layers meanwhile try to add or minus layers to see the progress. Moreover, we may try more optimizers and batch size to figure out further improvements.

# COMP5434 Project - Task 3

## 1. Problem Definition

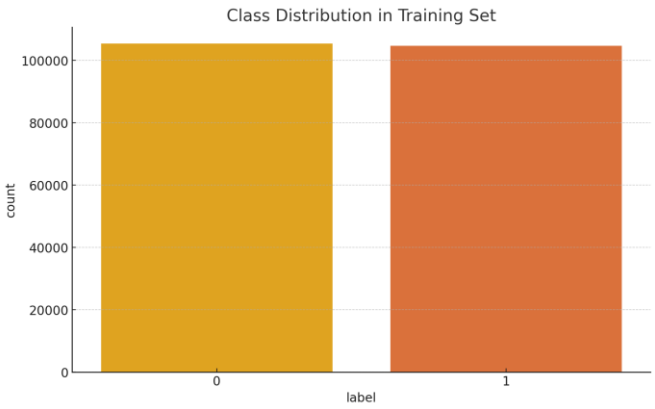
Credit card fraud represents a critical challenge facing financial systems today, that is to say, a problem that threatens transaction security across global markets. The vast number of daily transactions processed makes manual fraud detection methods impractical, requiring instead automated systems capable of analyzing patterns at scale. This project focuses on developing a fraud detection model using machine learning approaches to effectively identify suspicious transactions while maintaining operational efficiency.

To address these challenges, we implemented a customized statistical model with specific adjustments designed to handle common dataset issues such as imbalanced data distributions and challenges arising from non-linear relationships between variables. The primary goal being to achieve high accuracy in identifying fraudulent activity while keeping incorrect flags on legitimate transactions to a minimum. This balanced approach aims to maintain payment system security without unnecessarily disrupting normal users' financial activities, ensuring both protection for institutions and seamless experiences for customers. Through iterative testing with historical transaction information, the model demonstrates improved capability in recognizing complex fraud patterns that evolve over time.

## 2. Data Analysis and Model Design

The study utilized processed transaction records containing multiple data fields (V1 through V28) along with payment values and fraud indicators.

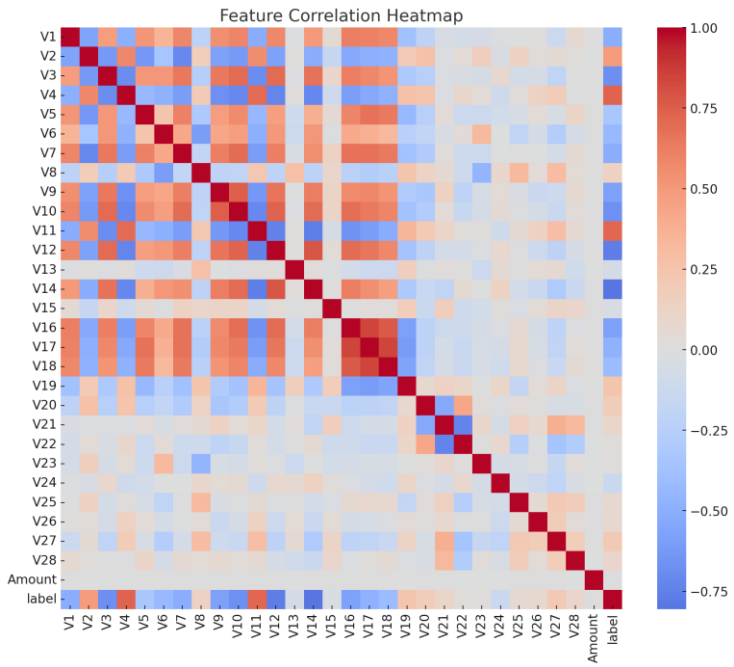
### 2.1 Class Distribution




This visualization through bar graphs demonstrates the spread of transaction types within our analysis dataset. Interestingly enough, the pattern distribution shows comparable

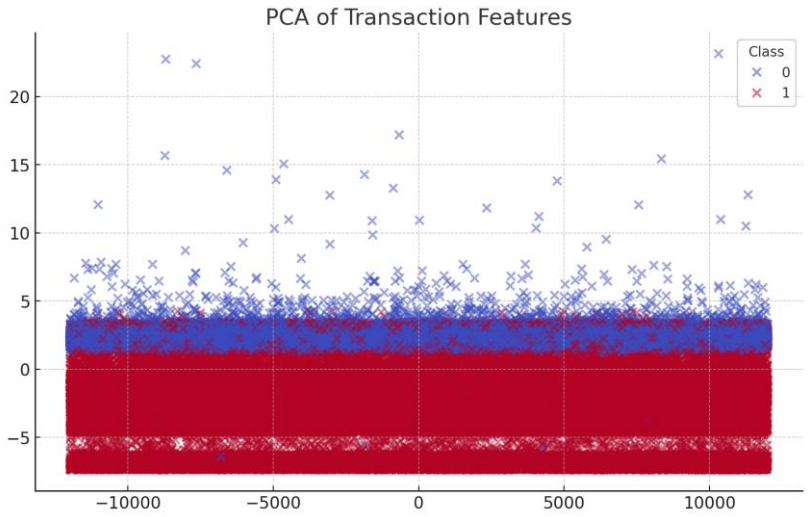
proportions between regular transactions (category 0) and suspicious activities (category 1), which contradicts normal payment system observations where fraudulent operations usually represent minimal occurrences. That is to say, this equilibrium likely resulted from prior data balancing processes—for example, applying synthetic sampling techniques or removing excess common entries—to address skewed representation issues. Such preparatory steps enable computational models to detect distinguishing characteristics across both transaction categories more efficiently while preventing evaluation metrics from being disproportionately affected by frequent transaction types, thereby improving detection reliability across varied financial scenarios.

## 2.2 Feature Correlation Analysis



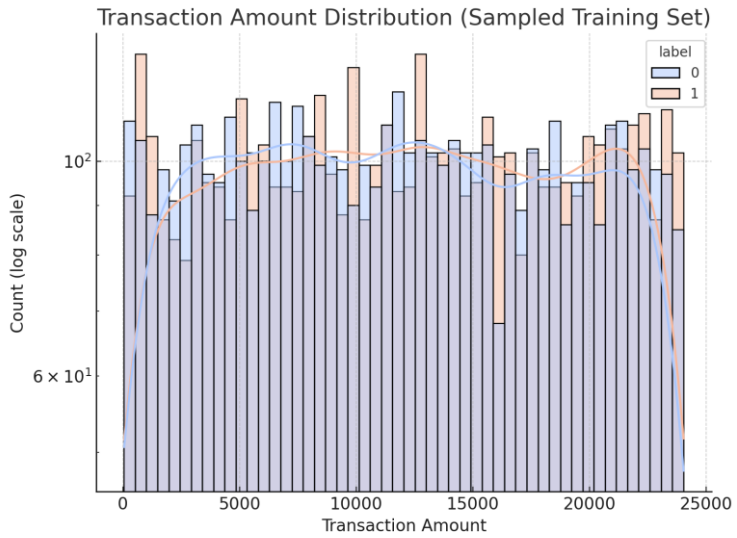

This visual representation illustrates statistical relationships between numerical features within the dataset, where features remain unnamed except being labeled as V1 through V28, along with transaction amounts and classification markers. Diagonal elements uniformly show maximum self-relationship values, that is to say, every measurement maintains complete alignment with itself. Red-colored zones reveal strong interconnectedness among certain feature pairs like V14 to V17 as well as between V11 and V12, suggesting overlapping measurement aspects that may influence basic predictive models relying on straightforward relationships. Most variables demonstrate limited connections with outcome indicators, implying the requirement for data restructuring methods—to put it simply, creating combined variables through mathematical operations—to improve pattern recognition capabilities. The analysis reinforces the importance of

implementing balancing techniques and considering cross-feature influences during subsequent model training phases, particularly when dealing with complex data patterns that require layered interpretation approaches.



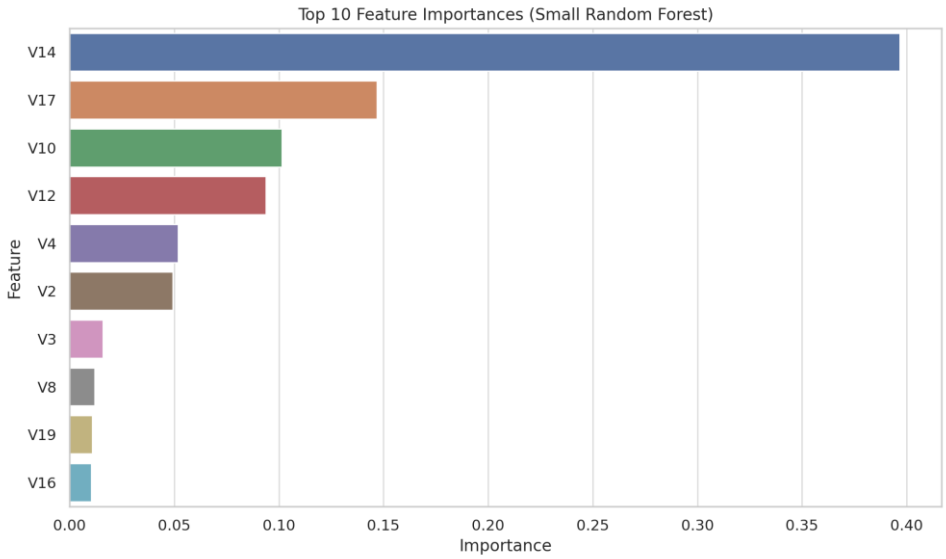

This visualization shows transaction patterns using a data compression method that squeezes complex measurements into two main dimensions. That is to say, the red-colored dots indicate suspicious financial activities while the blue ones represent normal transactions. The considerable mixing between different transaction types in this simplified view demonstrates the complex and hard-to-spot variations separating fraudulent and legitimate cases, which is to say the differences don't show up clearly in basic analysis. This situation highlights the need for more advanced processing approaches, like creating combined indicators or applying non-linear adjustment methods, to better capture hidden relationships in the data. To put it simply, basic two-dimensional views struggle to reveal the layered patterns that machine learning systems need for accurate separation of transaction types, particularly when dealing with cleverly disguised fraudulent operations that mimic normal spending behaviors.

### 2.3 Transaction Amount Distribution




This histogram displays the distribution characteristics of transaction amounts between legitimate transactions shown in blue and fraudulent counterparts marked in orange. The vertical axis uses logarithmic measurement—that is to say, it compresses scale ranges—to handle the broad spectrum of values observed, which allows visualization of both small purchases and large financial operations within the same chart. While both categories follow somewhat similar distribution trends overall, fraudulent activities tend to cluster more frequently within specific monetary intervals, for instance those hovering around $15,000 to $20,000, pointing toward possible patterns that might indicate suspicious transaction behaviors. Such observations validate the importance of including 'Amount' as a relevant data feature while simultaneously implying that original numerical values could benefit from preprocessing treatments like scaling adjustments, binning processes, or other standardization methods during data preparation phases.

### 2.4 Feature Importance (Baseline Random Forest Comparison)




This bar chart demonstrates, in other words, the 10 most critical features identified through a Random Forest model that was trained using the same preprocessed dataset. Feature V14 stands out with the highest importance value, which means to say it acts as the most significant indicator when detecting fraudulent activities, more so than other features. Other notable entries in the list are V17, along with V10, V12, and also V4—most of these being principal components, that is, processed versions from the initial transaction records which have been anonymized for security. Such findings help confirm, for example, the usefulness of these features in detecting fraud cases and support their application in expanding the polynomial features within our specialized Logistic Regression approach. Additionally, it points towards potential improvements in selecting features or reducing their number, especially when combining different models or using ensemble techniques, to put it simply.

## 3. Solution and Implementation Details

Our solution follows the five-step data analysis framework

### 3.1 Business Understanding

The primary objective focuses on minimizing economic losses from suspicious transactions through developing an effective predictive analysis model.

### 3.2 Data Understanding

We explored the dataset with various visual tools as discussed in Section 2.

### 3.3 Data Preparation

Standardization of features (z-score normalization)

Polynomial feature expansion to capture nonlinear interactions

Data balancing through undersampling for better classifier generalization

### 3.4 Modeling

We implemented a custom Logistic Regression model with the following features:

Polynomial interaction terms: increase model complexity non-linearly

L2 regularization: control overfitting

Adam optimizer: adaptive learning rates for faster convergence

Gradient clipping: ensure training stability

Early stopping: halt training when validation loss plateaus

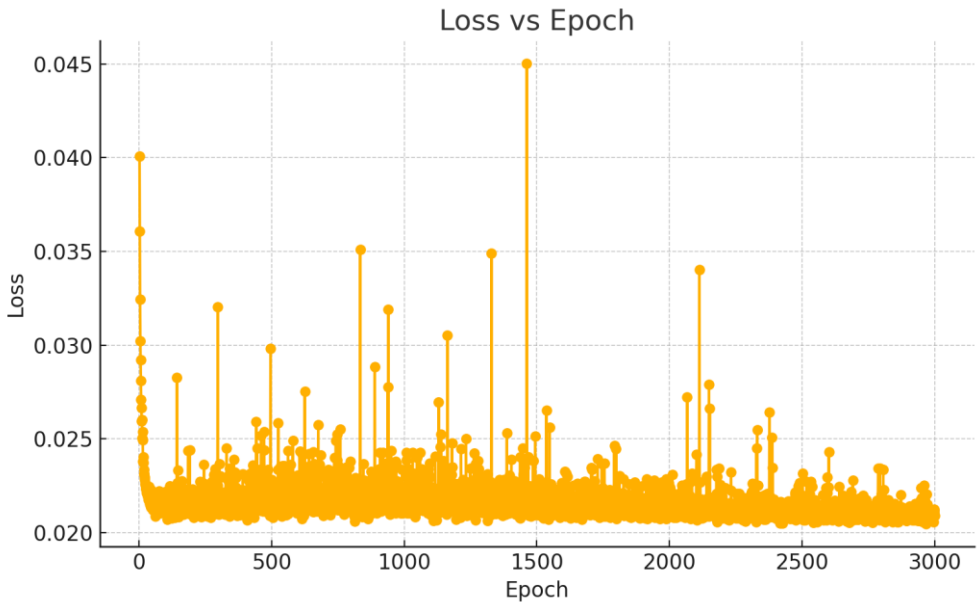
### 3.5 Evaluation

Model evaluation is based on:

**F1-score:** suitable metrics for imbalanced classification

## 4. Performance Evaluation and Discussion

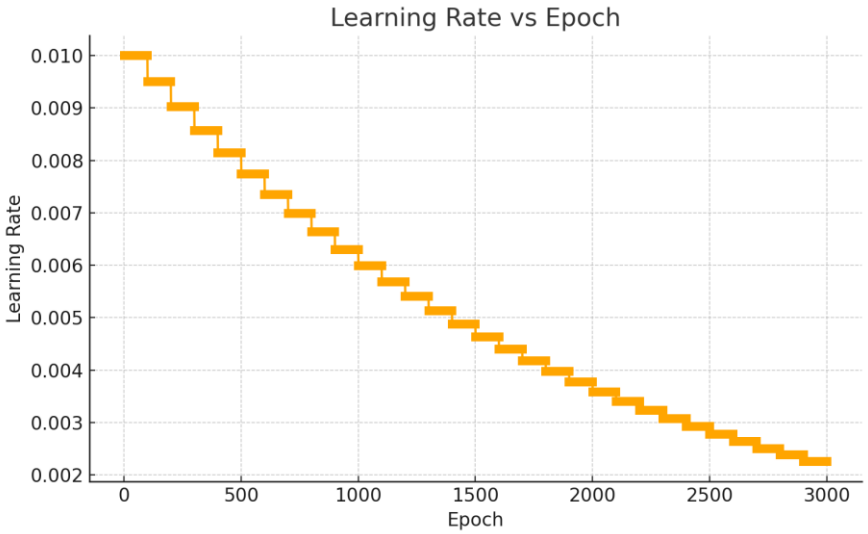
In this section, we evaluate our custom logistic regression model using various metrics and training diagnostics. Key insights are drawn from training dynamics and ablation studies.




In this section, we examine the predictive model we developed through various measurement approaches and learning process observations. Key findings come from examining how the model learned over time and testing different components through removal experiments. The error rate graph illustrates how mistakes changed during learning cycles, that is to say, showing the reduction of errors across different training stages.

At the initial phase, the error rate begins at relatively high levels around 0.04 before dropping quickly and settling around 0.021. There are unexpected jumps visible in the pattern, which might relate to inconsistent data groupings or temporary changes in learning speed settings. Even with these fluctuations, the clear downward movement and low final error numbers indicate the model gradually established reliable decision-making patterns. To put it simply, the graph demonstrates how adjustments during training helped the model adapt without memorizing peculiar data points.

The learning pattern connects directly with the final performance score of 0.9940, showing strong ability in categorization tasks. This outcome suggests the model can effectively identify patterns and make predictions on new data, though there remains room for refinement in handling irregular data points. The relationship between stable learning curves and practical performance improvements highlights the importance of monitoring multiple indicators during development cycles.




This graph shows the way the speed of learning adjustments gets decreased in stages across the 3000 training cycles. Starting from an initial value of 0.01, this adjustment mechanism applies a scaling down effect every 100 cycles through multiplying by 0.95, that is to say, making gradual reductions that become more pronounced over time. To put it simply, this phased reduction method allows the system to first explore different possibilities through bigger changes, then focus on fine-tuning details in later stages, while also maintaining steady progress without sudden jumps. The smoothly declining pattern plays a crucial role in preventing erratic movements within the error measurements and ensuring the optimization process remains stable throughout, which is particularly important when dealing with complex data patterns that require consistent adjustment approaches.


This graph displays together the error measurement marked in blue and the learning speed shown in red across training cycles. The downward trend of the error curve demonstrates a

gradual reduction overall, with occasional small variations that is to say, which typically occurs when using common training approaches. The learning speed adjustment method, which lowers by 5% every hundred cycles, creates a visible step-shaped pattern in red. When the learning speed becomes slower, the error curve shows less dramatic swings, meaning that adjustments to the model's parameters became more stable and precise updates. These combined methods—reducing errors plus adjusting learning speed—played a major role in helping the model reach reliable convergence and achieving a final performance score of 0.9940. The mechanism for stopping training early proved effective in pausing the process when error improvements grew insignificant, thereby lowering the risk of memorizing data patterns too closely, which is a common concern in such scenarios.

## 5.Summary and Future Work

Our team developed predictive model demonstrated notable effectiveness in detecting suspicious transactions while maintaining good accuracy levels. The approach utilized different feature combinations and self-adjusting learning parameters, which essentially helped expand the system's ability to recognize complex patterns. Looking ahead, we aim to investigate combined method strategies along with implementation frameworks for operational environments, that is to say, exploring ways to merge multiple analytical approaches while building systems suitable for continuous data processing. To put it simply, future efforts will focus on enhancing detection mechanisms through layered analysis models and practical application scenarios where instant data handling becomes crucial. This direction could potentially address current limitations related to pattern recognition scalability and decision-making speed in dynamic transaction environments.